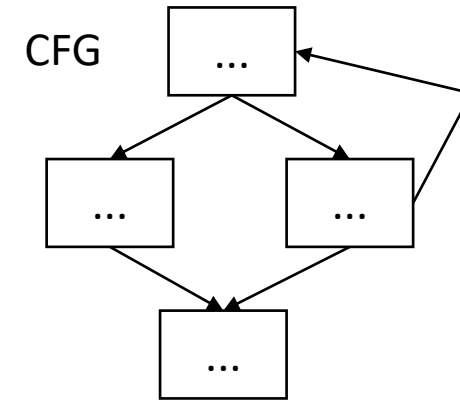
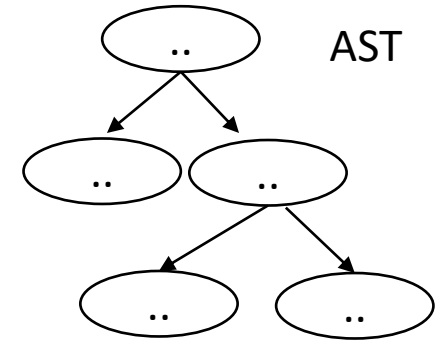


CSE110A: Compilers

Topics:

- *Module 3: Intermediate representations*
 - *Type Systems : Inference, Checking*
 - *Error Checking and Type Conversions: 38*
 - *Functions (72)*
 - *More on Types (80)*



3 address code

```
store i32 0, ptr %2
%3 = load i32, ptr %1
%4 = add nsw i32 %3, 1,
store i32 %4, ptr %1
%5 = load i32, ptr %2
```

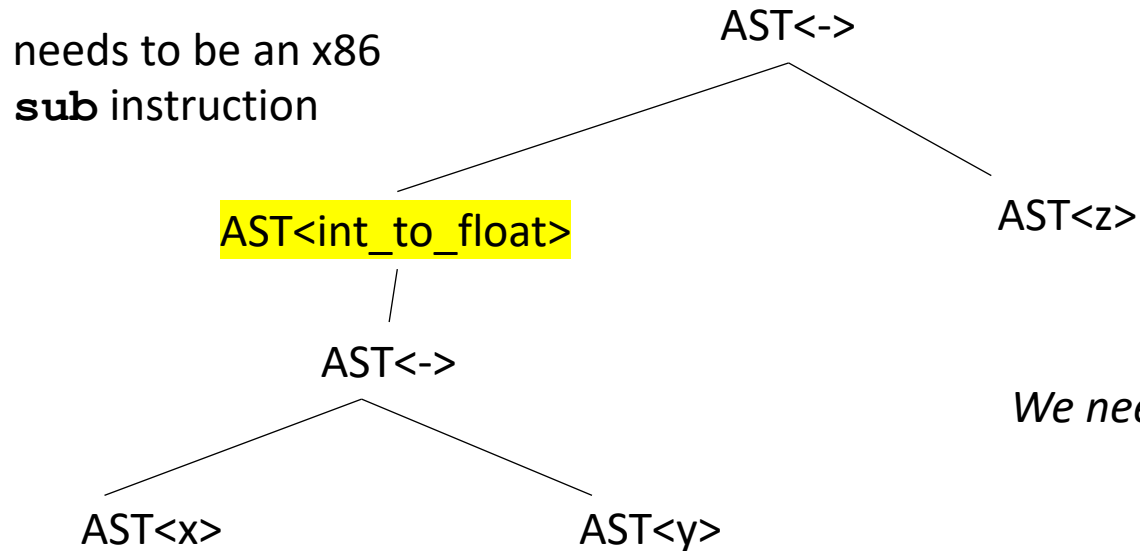
Topic: Type Systems

Evaluate an AST by doing a post order traversal

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      |  ""
```

```
int x;
int y;
float z;
float w;
w = x - y - z
```

needs to be an x86
subss instruction



We need to make sure our operands are in the right format!

Type systems

- Given a language a type system defines:
 - The primitive (base) types in the language
 - How the types can be converted to other types
 - implicitly or explicitly
 - How the user can define new types

Type checking and inference

- Check a program to ensure that it adheres to the type system

Especially interesting for compilers as a program given in the type system for the input language must be translated to a type system for lower-level program

Type systems

Considerations:

- Base types:
 - ints
 - chars
 - strings
 - floats
 - bool
- How to combine types in expressions:
 - int and float?
 - int and char?
 - int and bool?

Type checking

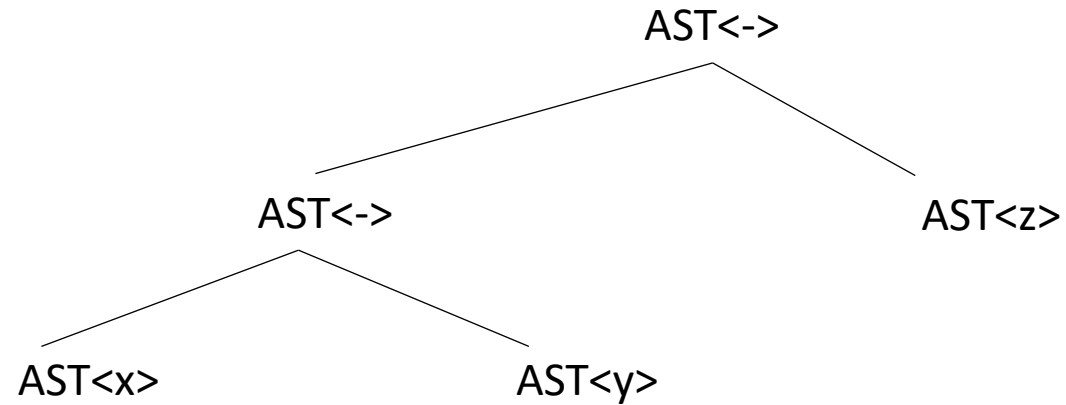
Two components

- Type inference
 - Determines a type for each AST node
 - Modifies the AST into a type-safe form
- Catches type-related errors

Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x - y - z
```

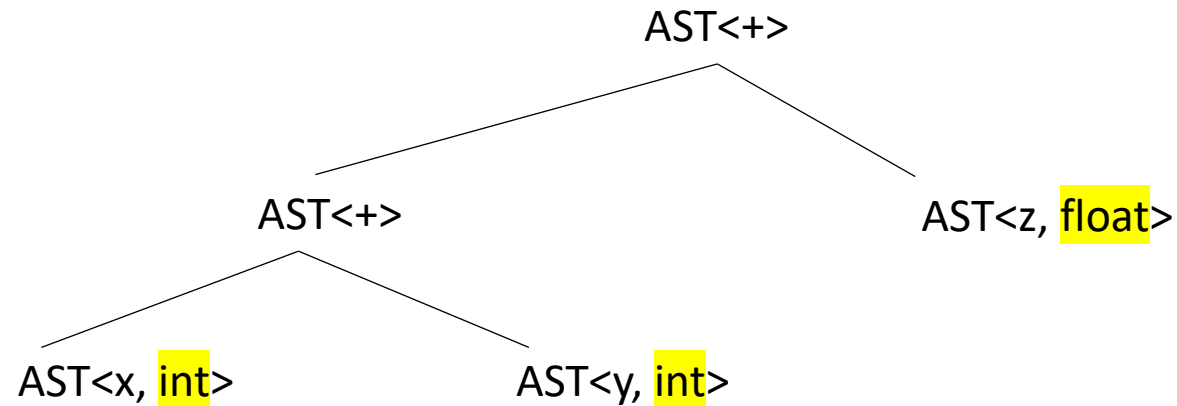
each node additionally gets a type



Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x - y - z
```

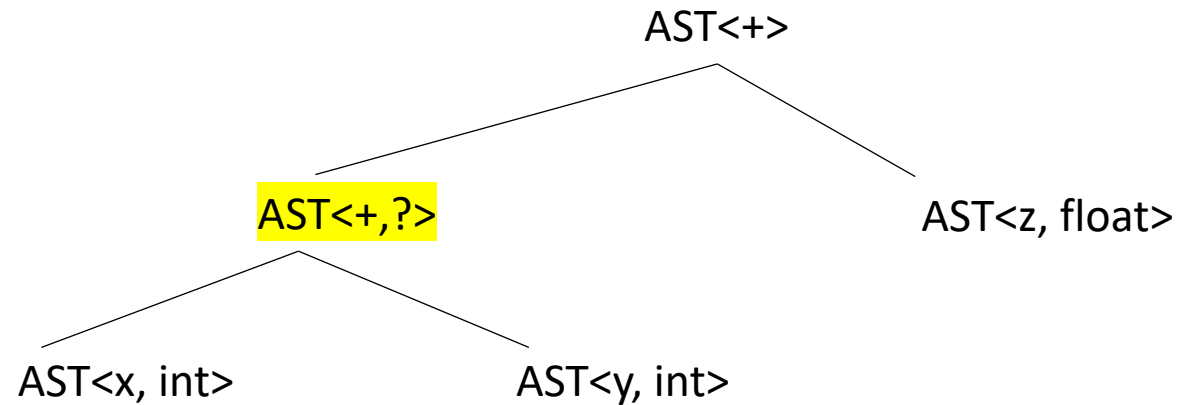
*each node additionally gets a type
we can get this from the symbol table for the leaves or based
on the input (e.g. 5 vs 5.0)*



Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x - y - z
```

How do we get the type for this one?



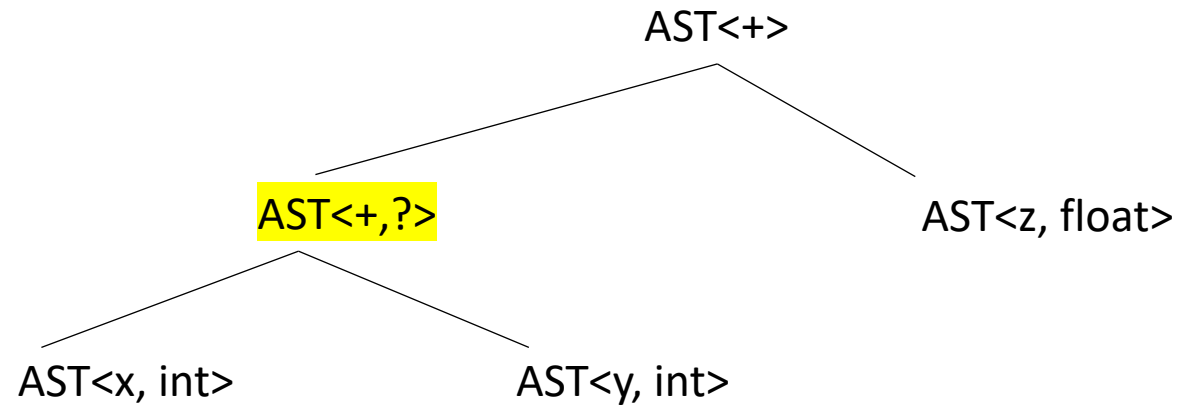
Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x - y - z
```

How do we get the type for this one?

inference rules for addition:

first	second	result
int	int	int
int	float	float
float	int	float
float	float	float



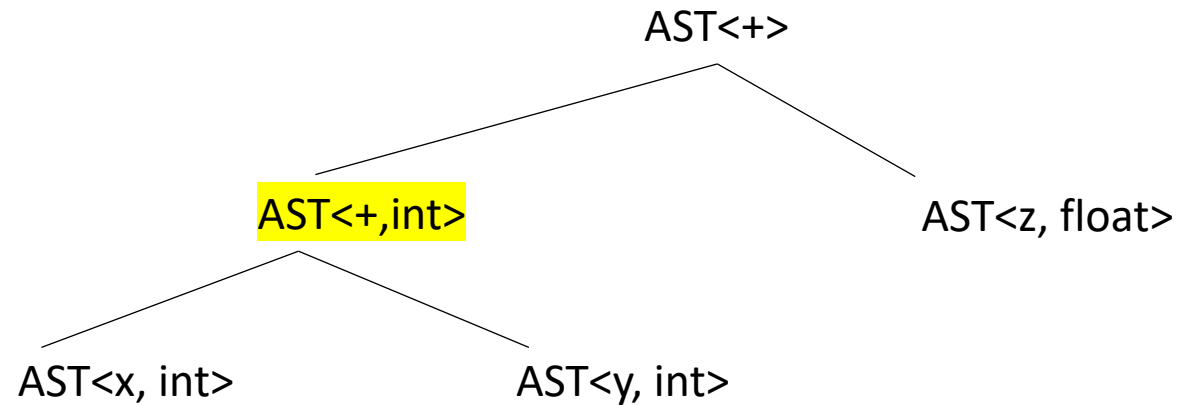
Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x - y - z
```

How do we get the type for this one?

inference rules for addition:

first	second	result
int	int	int
int	float	float
float	int	float
float	float	float



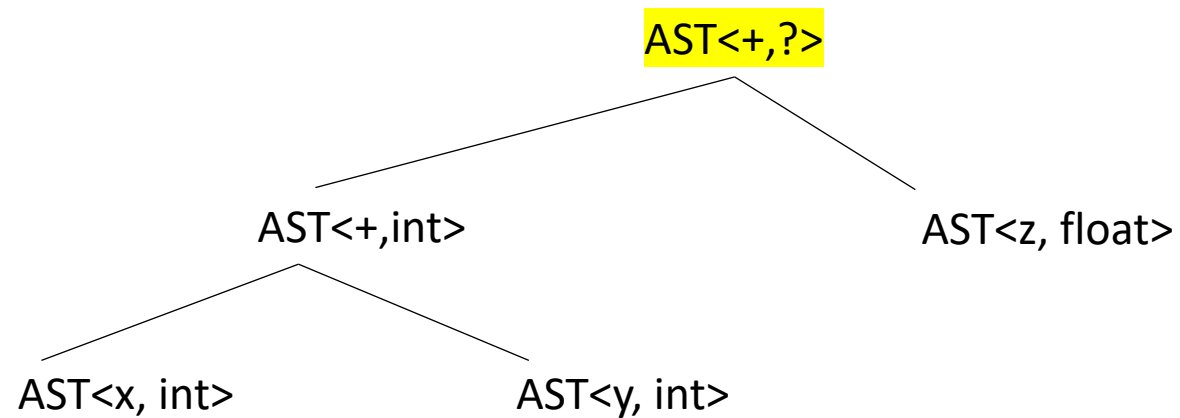
Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x - y - z
```

How do we get the type for this one?

inference rules for addition:

first	second	result
int	int	int
int	float	float
float	int	float
float	float	float



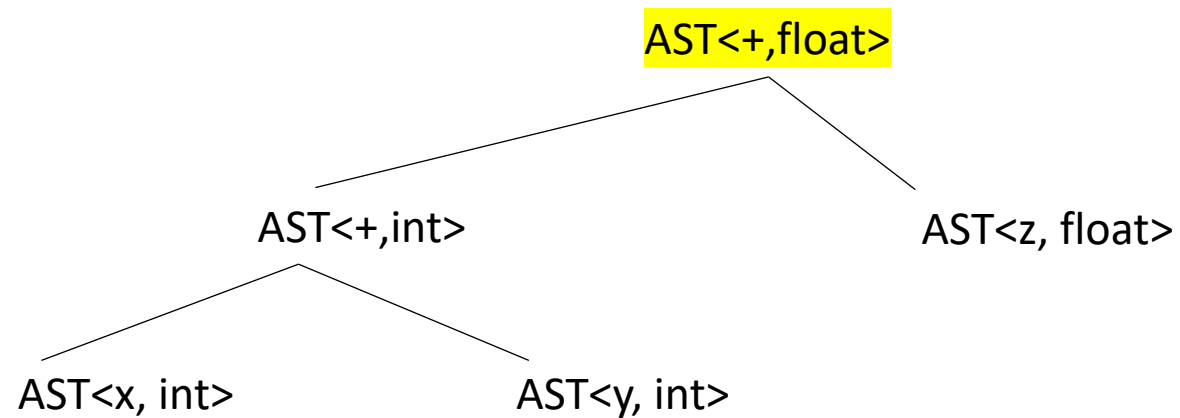
Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x - y - z
```

How do we get the type for this one?

inference rules for addition:

first	second	result
int	int	int
int	float	float
float	int	float
float	float	float



Type checking on an AST

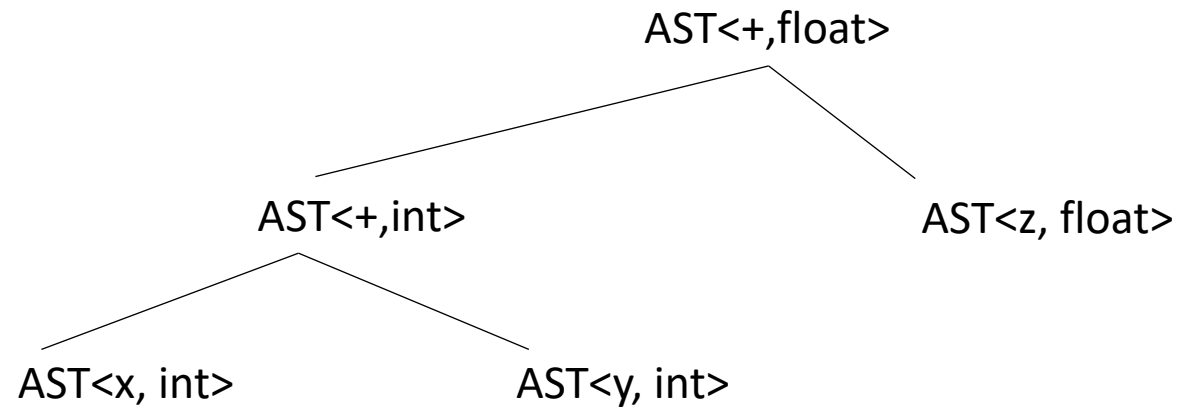
```
int x;  
int y;  
float z;  
float w;  
w = x + y + z
```

How do we get the type for this one?

inference rules for addition:

first	second	result
int	int	int
int	float	float
float	int	float
float	float	float

what else?



Type checking on an AST

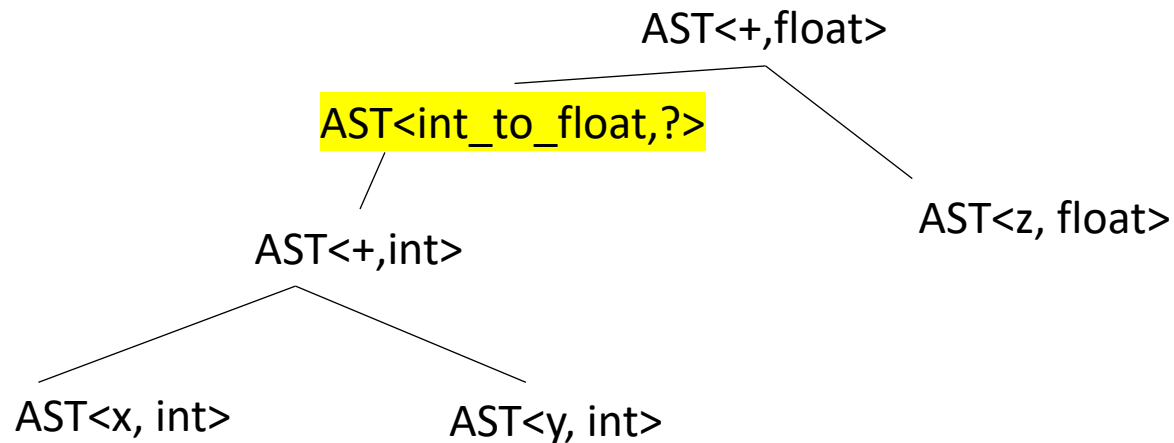
```
int x;  
int y;  
float z;  
float w;  
w = x + y + z
```

How do we get the type for this one?

inference rules for addition:

first	second	result
int	int	int
int	float	float
float	int	float
float	float	float

what else? need to convert the int to a float



```
class ASTNode():
    def __init__(self):
        pass
```

```
class ASTLeafNode(ASTNode):
    def __init__(self, value):
        self.value = value
```

```
class ASTNumNode(ASTLeafNode):
    def __init__(self, value):
        super().__init__(value)
```

```
class ASTIDNode(ASTLeafNode):
    def __init__(self, value):
        super().__init__(value)
```

```
class ASTBinOpNode(ASTNode):
    def __init__(self, l_child, r_child):
        self.l_child = l_child
        self.r_child = r_child
```

```
class ASTPlusNode(ASTBinOpNode):
    def __init__(self, l_child, r_child):
        super().__init__(l_child, r_child)
```

```
class ASTMultNode(ASTBinOpNode):
    def __init__(self, l_child, r_child):
        super().__init__(l_child, r_child)
```


Enum for types

```
from enum import Enum

class Types(Enum):
    INT = 1
    FLOAT = 2
```

Now we need to set the types for the leaf nodes

Our base AST Node needs a type

```
class ASTNode():
    def __init__(self):
        self.node_type = None
        pass

    def set_type(self, t):
        self.node_type = t

    def get_type(self):
        return self.node_type
```

Enum for types

```
from enum import Enum

class Types(Enum):
    INT = 1
    FLOAT = 2
```

Our base AST Node needs a type

```
class ASTNode():
    def __init__(self):
        self.node_type = None
        pass

    def set_type(self, t):
        self.node_type = t

    def get_type(self):
        return self.node_type
```

Now we need to set the types for the leaf nodes

```
class ASTNumNode(ASTLeafNode):
    def __init__(self, value):
        super().__init__(value)
        if is_int(value):
            self.set_type(Types.INT)
        else:
            self.set_type(Types.FLOAT)
```

Enum for types

```
from enum import Enum

class Types(Enum):
    INT = 1
    FLOAT = 2
```

Our base AST Node needs a type

```
class ASTNode():
    def __init__(self):
        self.node_type = None
        pass

    def set_type(self, t):
        self.node_type = t

    def get_type(self):
        return self.node_type
```

Now we need to set the types for the leaf nodes

```
class ASTNumNode(ASTLeafNode):
    def __init__(self, value):
        super().__init__(value)
        if is_int(value):
            self.set_type(Types.INT)
        else:
            self.set_type(Types.FLOAT)
```

```
class ASTIDNode(ASTLeafNode):
    def __init__(self, value, value_type):
        super().__init__(value)
        self.set_type(value_type)
```

Where can we get the value type for an ID?

Symbol Table

Say we are matched the statement:
`int x;`

- `SymbolTable ST;`

```

                                (TYPE, 'int') (ID, 'x')
declare_statement ::= TYPE ID SEMI
{
    eat(TYPE)
    id_name = self.to_match.value
    eat(ID)
    ST.insert(id_name, None)
    eat(SEMI)
}
```

*in homework 2 and 3 we didn't
record any information in the symbol
table*

Symbol Table

Say we are matched the statement:
`int x;`

- `SymbolTable ST;`

`declare_statement ::= (TYPE, 'int') TYPE (ID, 'x') ID SEMI`

`{`

`value_type = self.to_match.value`

`eat(TYPE)`

`id_name = self.to_match.value`

`eat(ID)`

`ST.insert(id_name, value_type)`

`eat(SEMI)`

`}`

*previously we weren't saving any
information about the ID*

record the type in the symbol table

Enum for types

```
from enum import Enum

class Types(Enum):
    INT = 1
    FLOAT = 2
```

Our base AST Node needs a type

```
class ASTNode():
    def __init__(self):
        self.node_type = None
        pass

    def set_type(self, t):
        self.node_type = t

    def get_type(self):
        return self.node_type
```

Now we need to set the types for the leaf nodes

```
class ASTNumNode(ASTLeafNode):
    def __init__(self, value):
        super().__init__(value)
        if is_int(value):
            self.set_type(Types.INT)
        else:
            self.set_type(Types.FLOAT)
```

```
class ASTIDNode(ASTLeafNode):
    def __init__(self, value, value_type):
        super().__init__(value)
        self.set_type(value_type)
```

Where can we get the value type for an ID?

But that doesn't get us here yet...

add the type at parse time

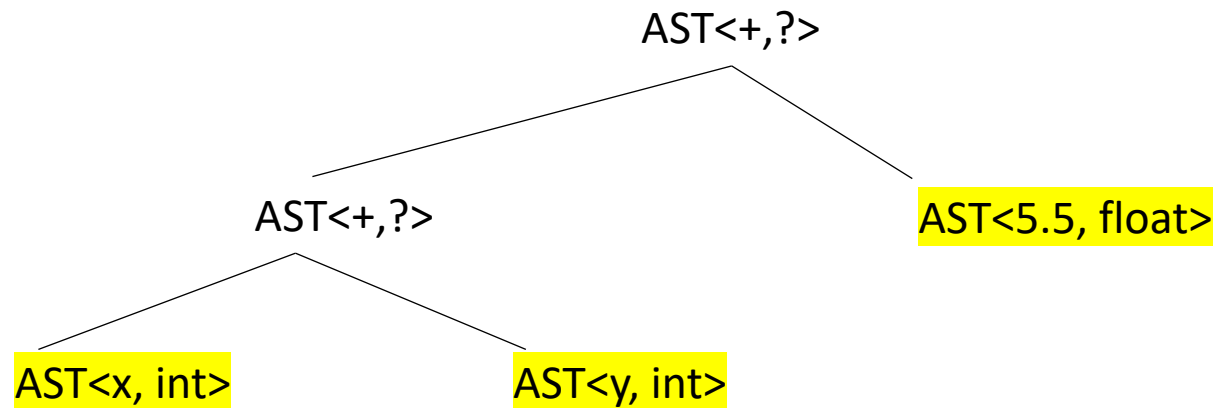
Unit ::= ID
NUM

```
def parse_unit(self, lhs_node):  
    # ... for applying the first production rule (ID)  
    value = self.next_word.value  
    # ... Check that value is in the symbol table  
    node = ASTIDNode(value, ST[value])  
    return node
```

Type inference

- We now have the types for the leaf nodes

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```

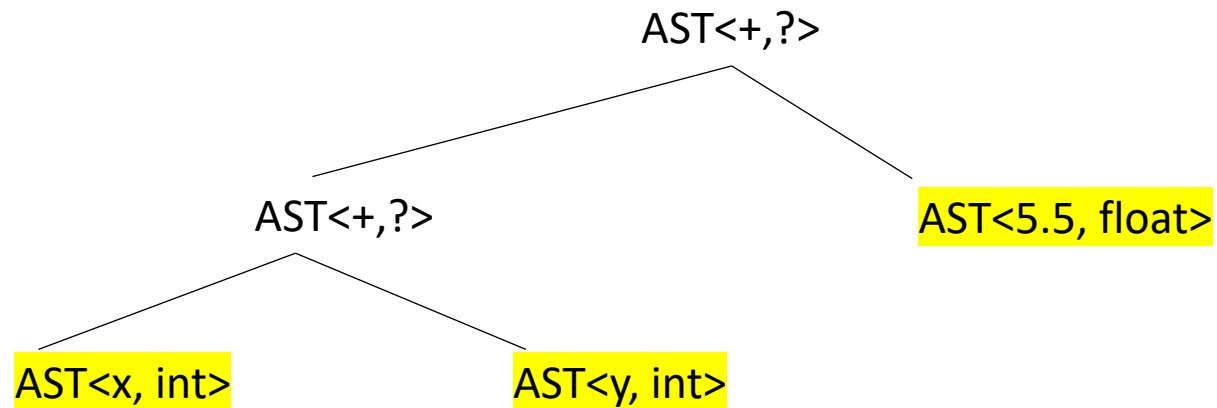


Type inference

- We now have the types for the leaf nodes

Next steps:

we do a post order traversal
on the AST and do a type inference



Type inference

`def type_inference(n):`

Given a node n: find its type and the types of any of its children

Type inference

```
def type_inference(n):
```

Given a node n: find its type and the types of any of its children

```
    case split on n:
```

```
        if n is a leaf node:
            return n.get_type()
```

base case

Type inference

```
def type_inference(n):
```

Given a node n: find its type and the types of any of its children

```
    case split on n:
```

```
        if n is a leaf node:
            return n.get_type()
```

```
        if n is a plus node:
            ...
```

Type inference

`def type_inference(n):`

Given a node n: find its type and the types of any of its children

case split on n:

if n is a leaf node:
 return n.get_type()

if n is a plus node:
 return lookup type from table

lookup the rule for plus

inference rules for plus

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
def type_inference(n):
```

Given a node n: find its type and the types of any of its children

```
    case split on n:
```

```
        if n is a leaf node:
            return n.get_type()
```

```
        if n is a plus node:
            lookup the rule for plus
            return lookup type from table
```

inference rules for plus

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float

but we're missing a few things

Type inference

`def type_inference(n):`

Given a node n: find its type and the types of any of its children

case split on n:

if n is a leaf node:
 return n.get_type()

*we need to make sure the
children have types!*

if n is a plus node:
 do type inference on children
 return lookup type from table

inference rules for plus

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
def type_inference(n):
```

Given a node n: find its type and the types of any of its children

```
    case split on n:
```

```
        if n is a leaf node:
            return n.get_type()
```

we should record our type

```
        if n is a plus node:
            do type inference on children
            t = lookup type from table
            set n type to t
            return t
```

inference rules for plus

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
def type_inference(n):
```

Given a node n: find its type and the types of any of its children

```
    case split on n:
```

```
        if n is a leaf node:
            return n.get_type()
```

```
        if n is a plus node:
            do type inference on children
            t = lookup type from table
            set n type to t
            return t
```

is this just for plus?

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
def type_inference(n):
```

Given a node n: find its type and the types of any of its children

```
    case split on n:
```

```
        if n is a leaf node:
            return n.get_type()
```

is this just for plus?

most language promote
types, e.g. ints to float for
expression operators

```
        if n is a plus node:
            do type inference on children
            t = lookup type from table
            set n type to t
            return t
```

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

def **type_inference**(n):

Given a node n: find its type and the types of any of its children

case split on n:

if n is a leaf node:
 return n.get_type()

is this just for plus?

most language promote
types, e.g. ints to float for
expression operators

if n is a bin op node:
 do type inference on children
 t = lookup type from table
 set n type to t
 return t

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
def type_inference(n):
```

```
    case split on n:
```

```
        if n is a leaf node:
            return n.get_type()
```

```
        if n is a bin op node:
            do type inference on children
            t = lookup type from table
            set n type to t
            return t
```

What about for assignments?

```
int x;
cout << (x = 5.5) << endl;
```

What does this return?

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
def type_inference(n):
```

```
    case split on n:
```

```
        if n is a leaf node:
            return n.get_type()
```

```
        if n is a bin op node:
            do type inference on children
            t = lookup type from table
            set n type to t
            return t
```

What about for assignments?

```
int x;
cout << (x = 5.5) << endl;
```

What does this return?

left	right	result
int	int	int
int	float	int
float	int	float
float	float	float

whatever the left is

Type checking

- Checking for errors

Type inference

`def type_inference(n):`

Given a node n: find its type and the types of any of its children

case split on n:

if n is a leaf node:
 return n.get_type()

we should record our type

if n is a plus node:
 do type inference on children
 t = lookup type from table
 if t is None:
 throw type exception
 set n type to t
 return t

inference rules for plus

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
def type_inference(n):
```

Given a node n: find its type and the types of any of its children

```
    case split on n:
```

```
        if n is a leaf node:
            return n.get_type()
```

we should record our type

```
        if n is a plus node:
            do type inference on children
            t = lookup type from table
            if t is None:
                throw type exception
            set n type to t
            return t
```

inference rules for plus

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float
string	int	None

like in Python

Type inference

What other examples would throw an error?

```
def type_inference(n):
```

Given a node n: find its type and the types of any of its children

```
    case split on n:
```

```
        if n is a leaf node:
            return n.get_type()
```

we should record our type

```
        if n is a plus node:
            do type inference on children
            t = lookup type from table
            if t is None:
                throw type exception
            set n type to t
            return t
```

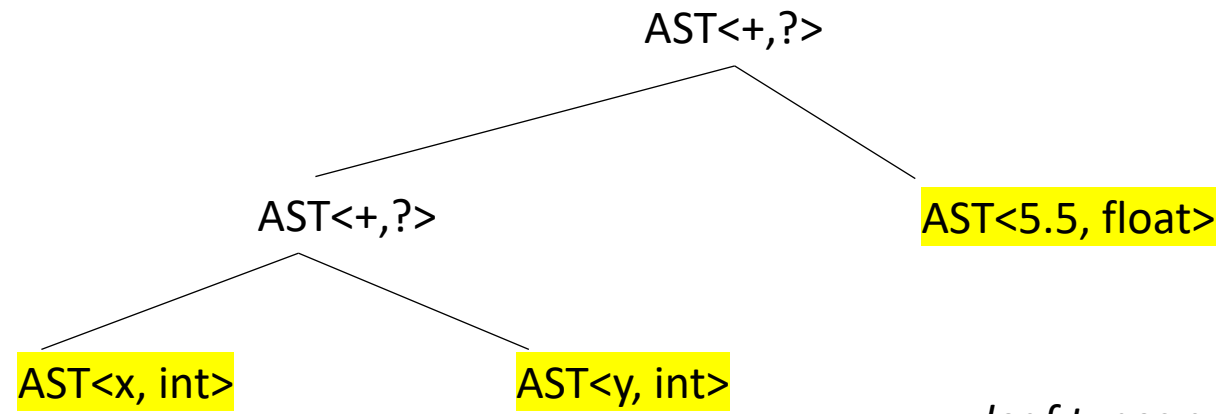
inference rules for plus

left	right	result
int	int	int
int	float	float
float	int	float
float	float	float
string	int	None

like in Python

Type inference

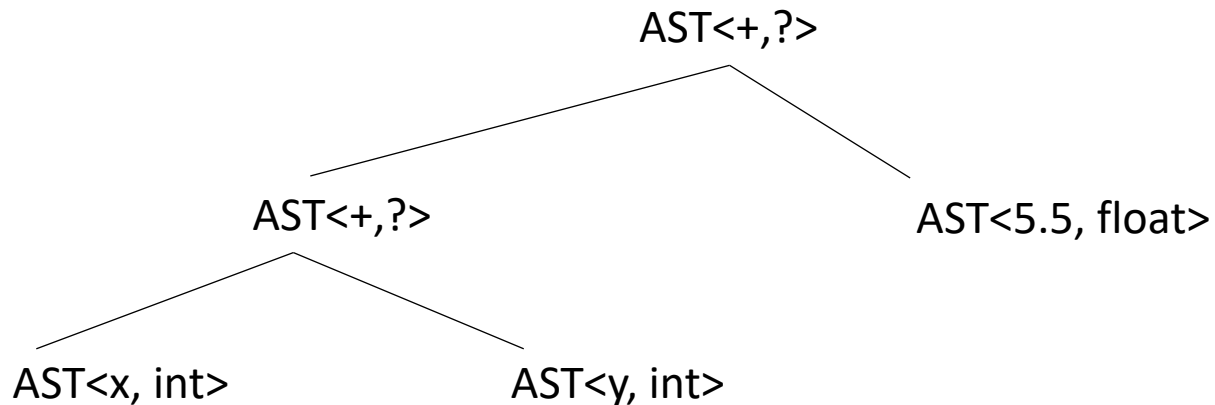
```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



leaf types are provided on construction

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:
```

```
            return n.get_type()
```

```
        if n is a bin op node:
```

```
            do type inference on children
```

```
            t = lookup type from table
```

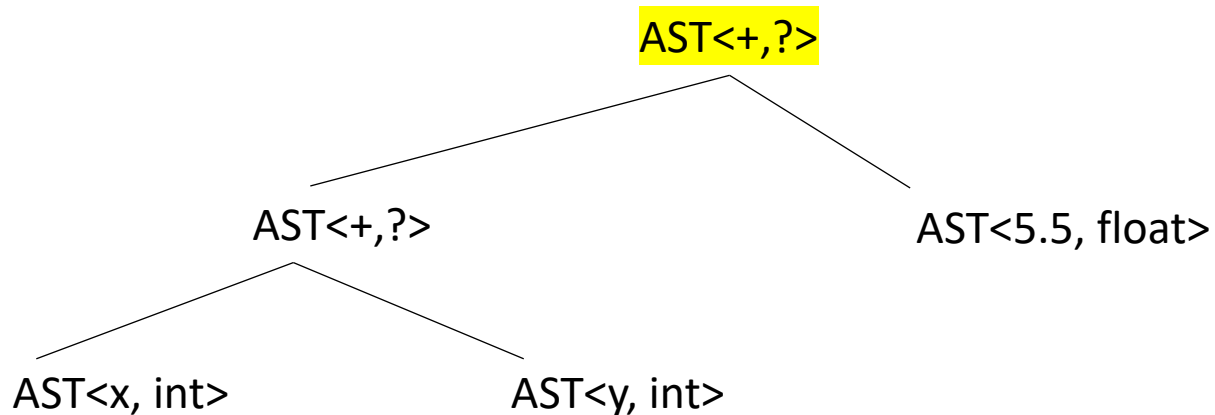
```
            set n type to t
```

```
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```

start on top



```
def type_inference(n):
```

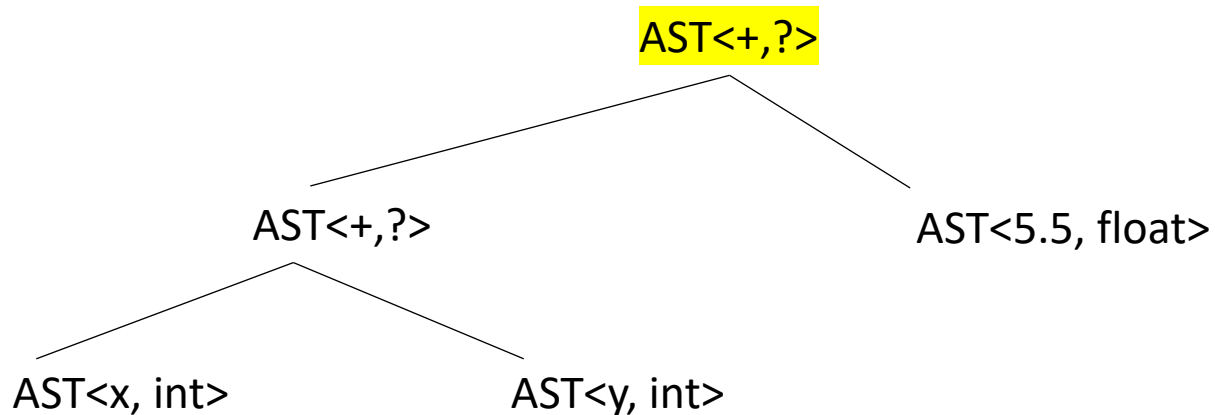
```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



it's a binary op

```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

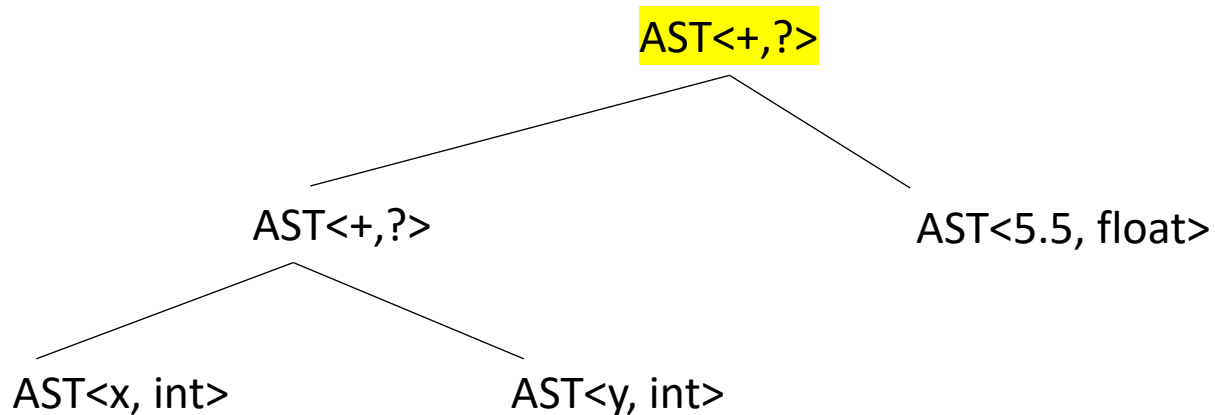
```
        if n is a bin op node:
```

```
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```

recursion



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:
```

```
            do type inference on children
```

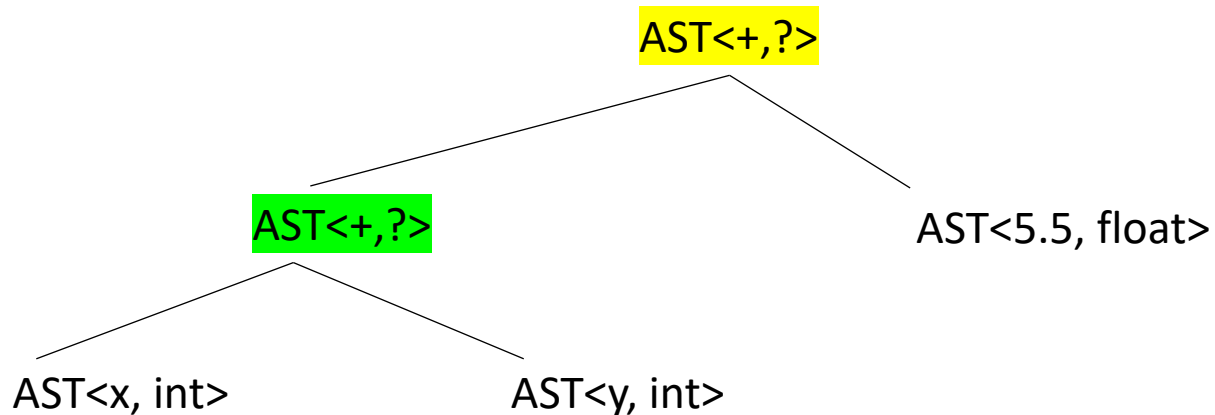
```
            t = lookup type from table
```

```
            set n type to t
```

```
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

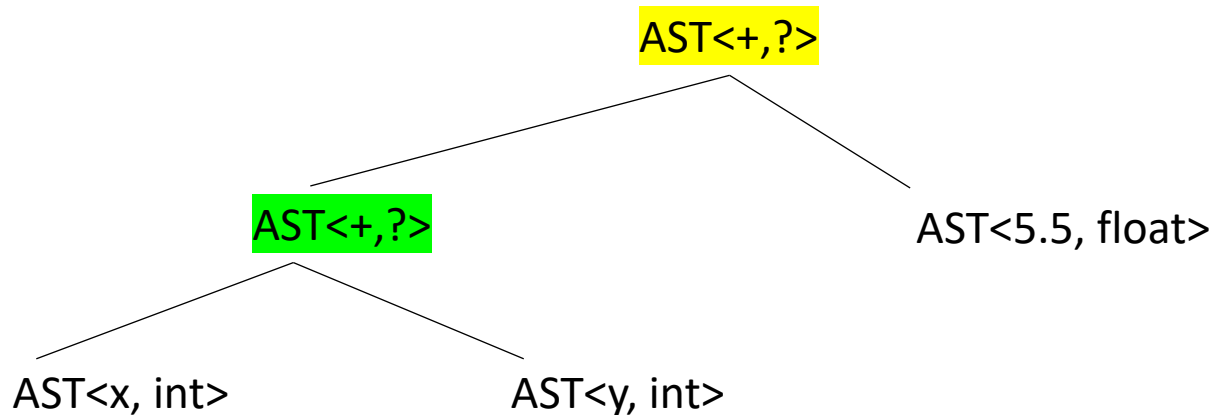
```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



it's a binary op

```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

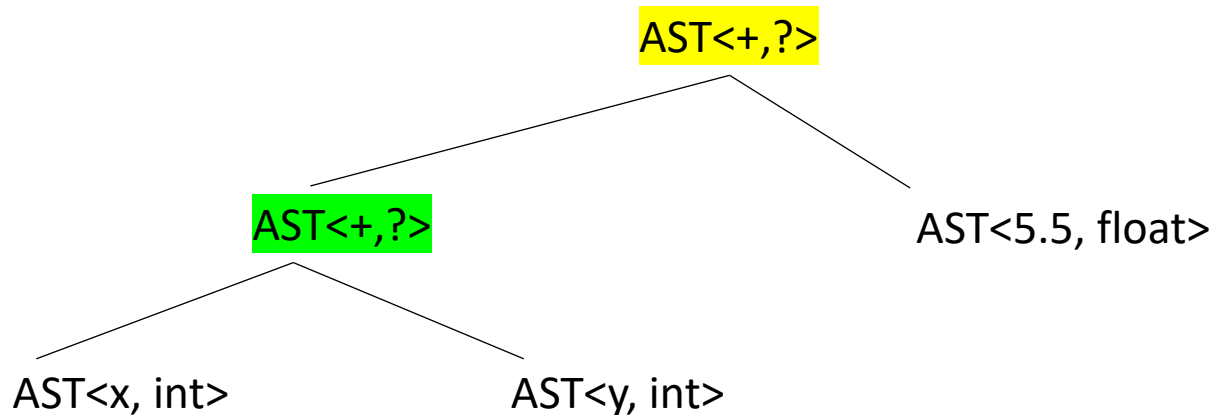
```
        if n is a bin op node:
```

```
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```


Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```

recursion



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:
```

```
            do type inference on children
```

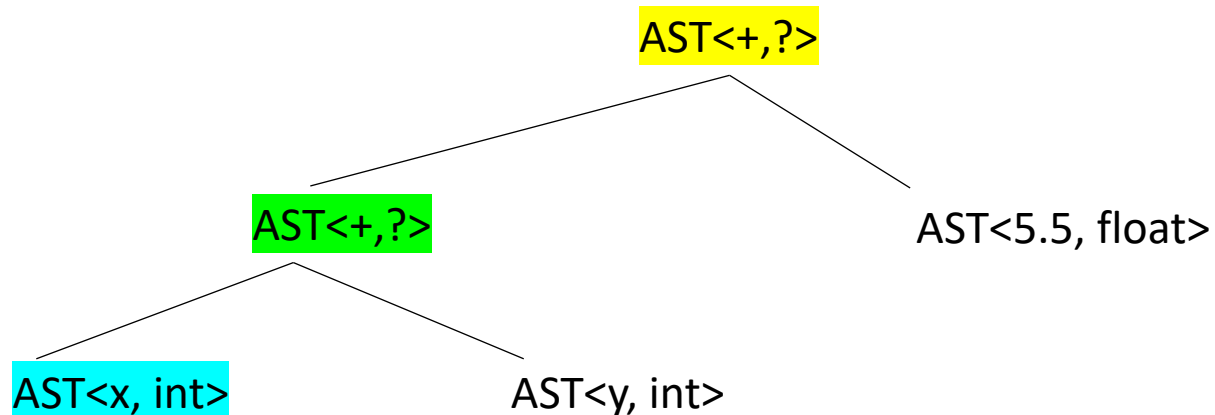
```
            t = lookup type from table
```

```
            set n type to t
```

```
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

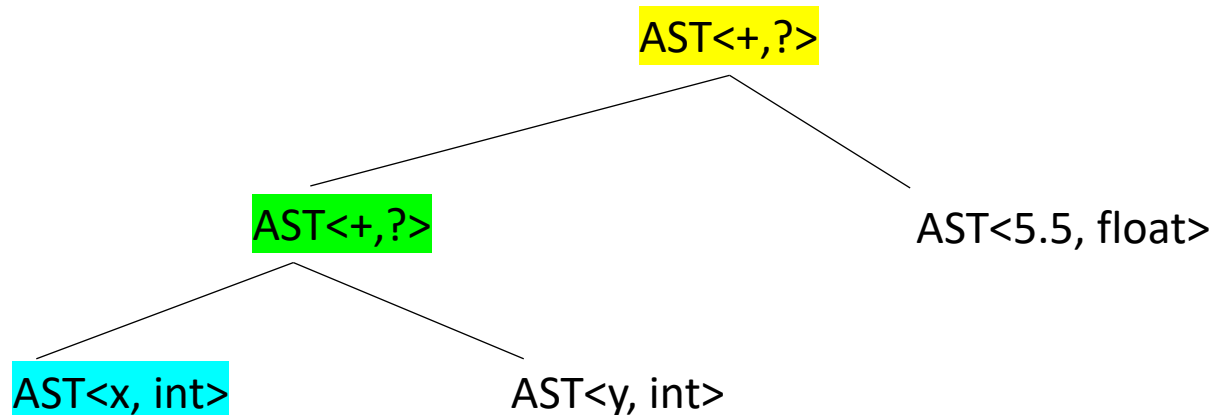
```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:
```

```
            return n.get_type()
```

```
        if n is a bin op node:
```

```
            do type inference on children
```

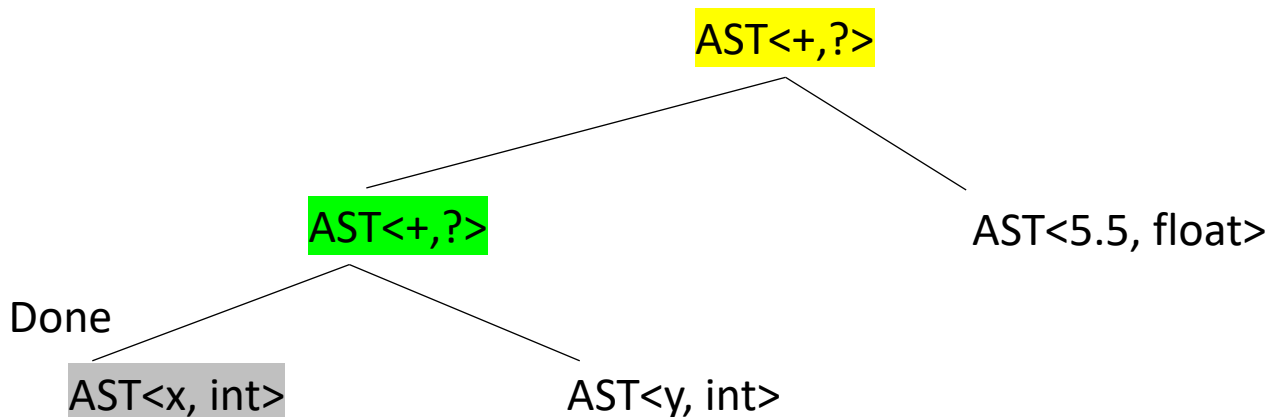
```
            t = lookup type from table
```

```
            set n type to t
```

```
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:
```

```
            do type inference on children
```

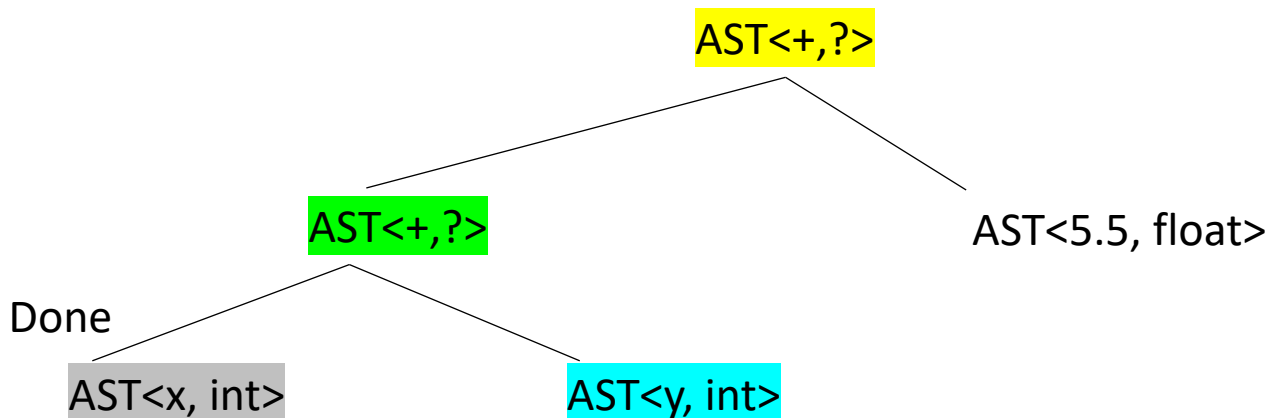
```
            t = lookup type from table
```

```
            set n type to t
```

```
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:
```

```
            return n.get_type()
```

```
        if n is a bin op node:
```

```
            do type inference on children
```

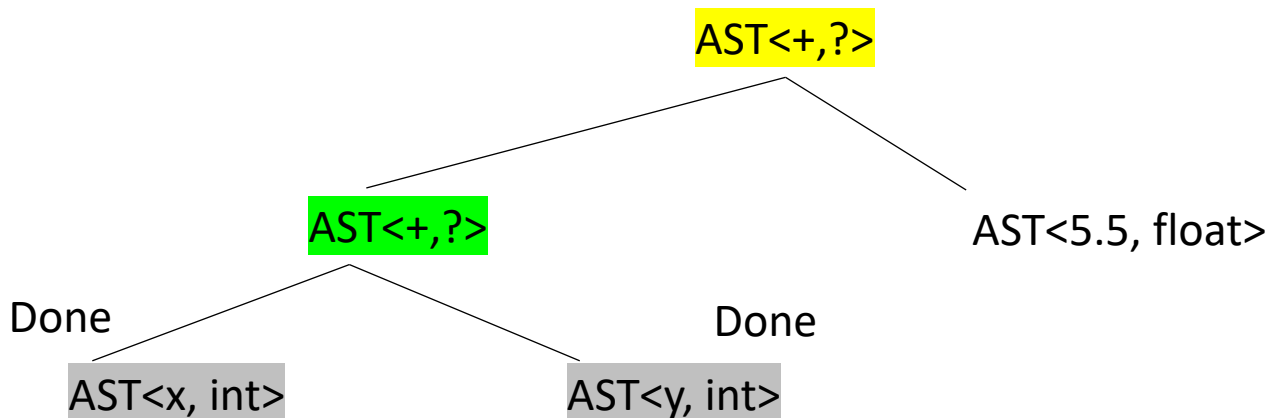
```
            t = lookup type from table
```

```
            set n type to t
```

```
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

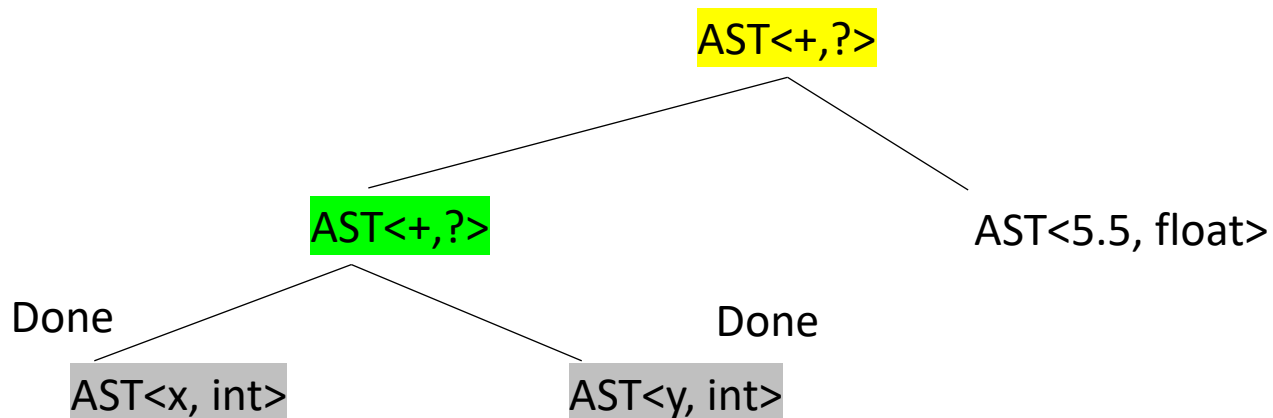
```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

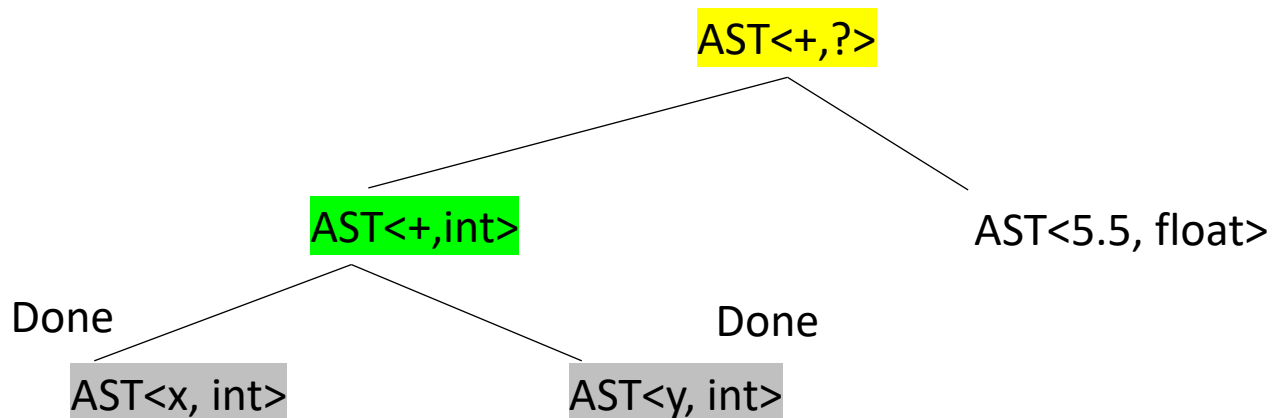
```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```

Table for **most** binary ops

left child	right child	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

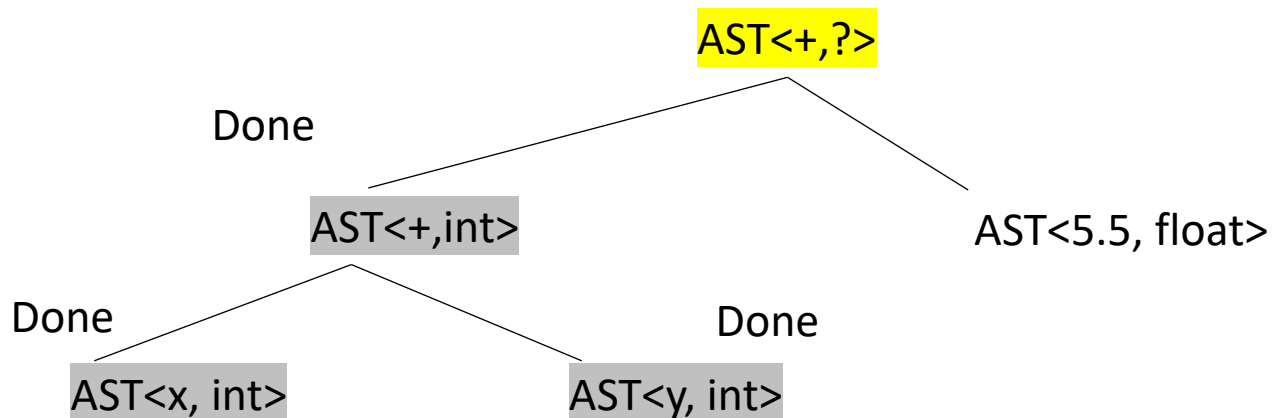
```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```

Table for **most** binary ops

left child	right child	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:
```

```
            do type inference on children
```

```
            t = lookup type from table
```

```
            set n type to t
```

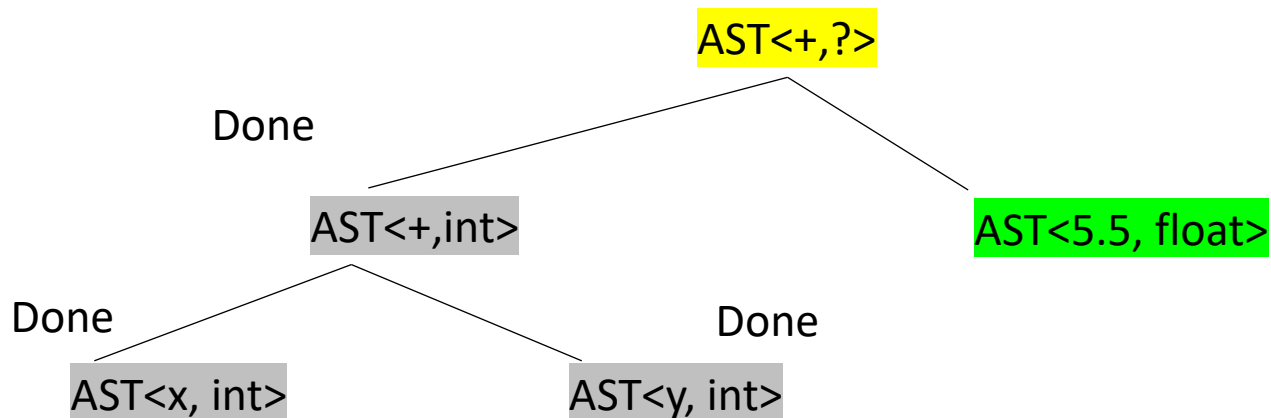
```
            return t
```

Table for **most** binary ops

left child	right child	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:
```

```
            return n.get_type()
```

```
        if n is a bin op node:
```

```
            do type inference on children
```

```
            t = lookup type from table
```

```
            set n type to t
```

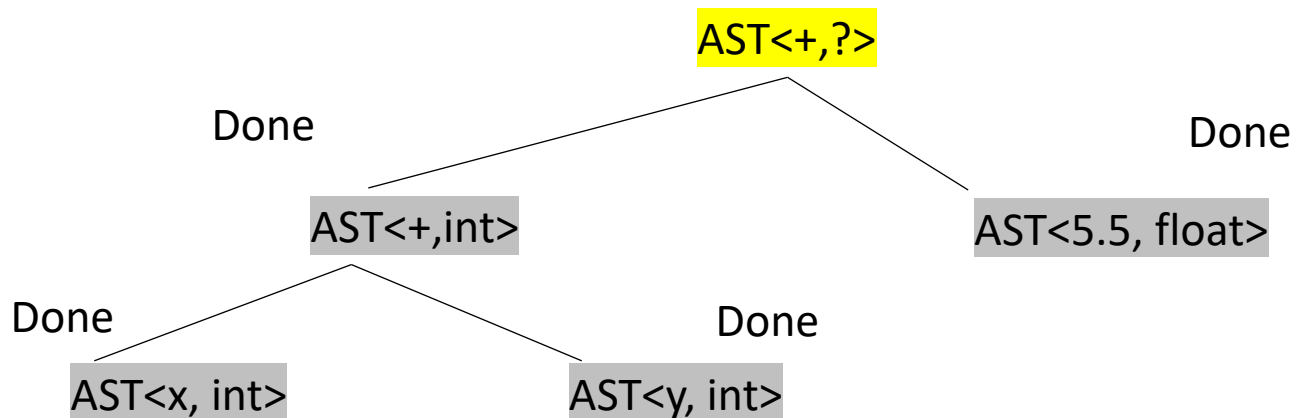
```
            return t
```

Table for **most** binary ops

left child	right child	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:
```

```
            do type inference on children
```

```
            t = lookup type from table
```

```
            set n type to t
```

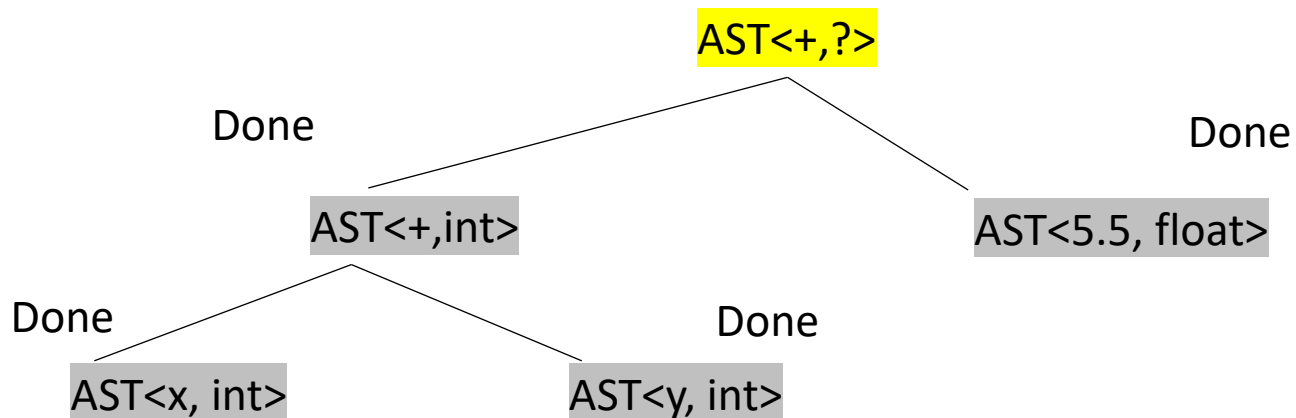
```
            return t
```

Table for **most** binary ops

left child	right child	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

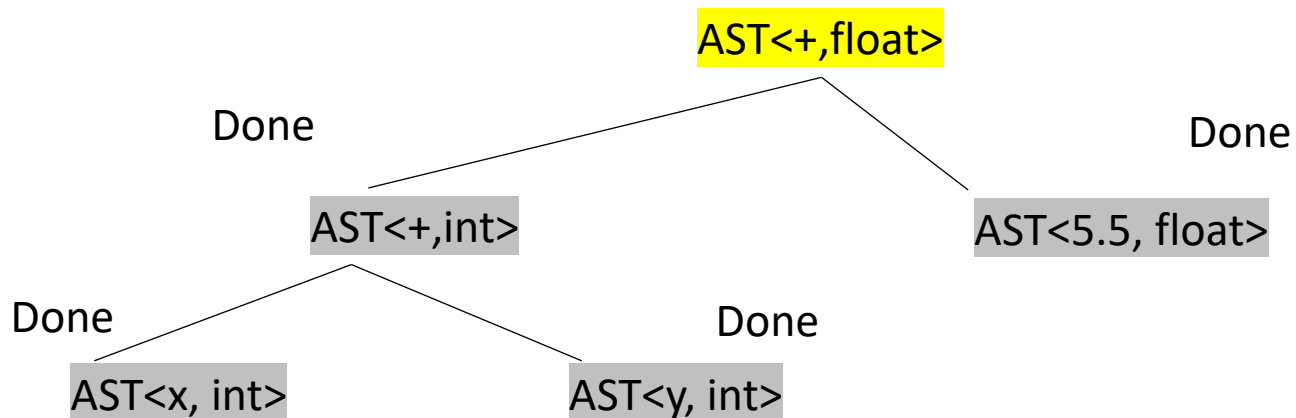
```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```

Table for **most** binary ops

left child	right child	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

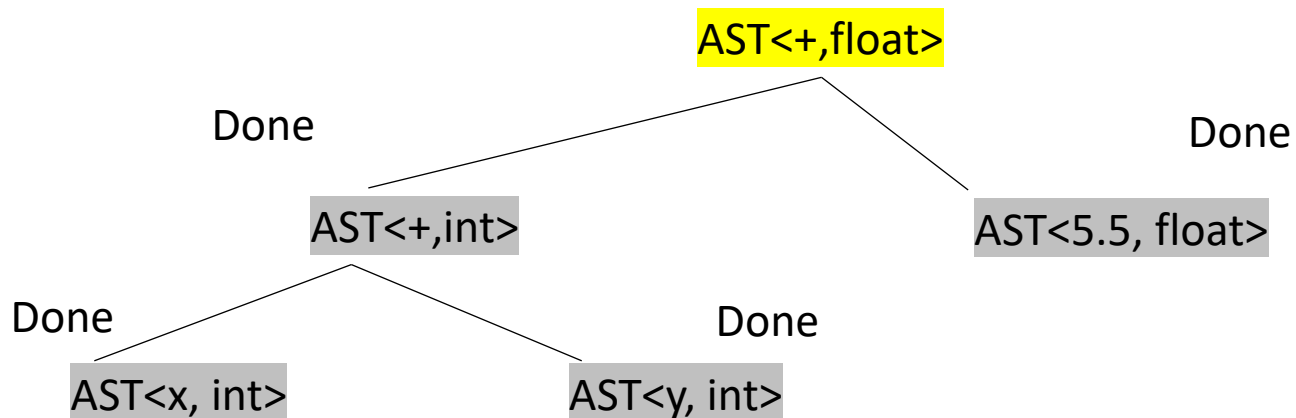
```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```

Table for **most** binary ops

left child	right child	result
int	int	int
int	float	float
float	int	float
float	float	float

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

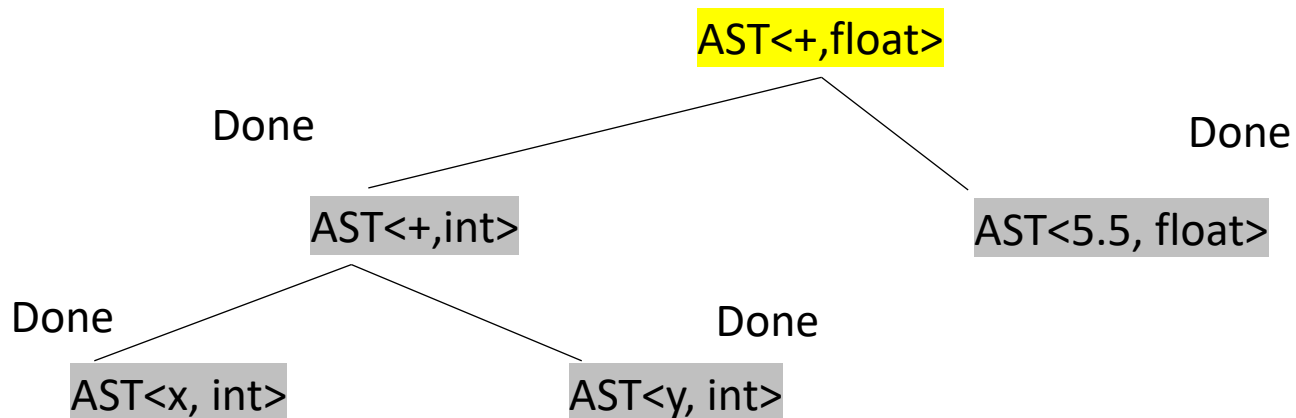
```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            return t
```

Are we done?

Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t
```

```
        do any required type conversions  
        return t
```

Are we done?

```
def type_conversion(n):
```

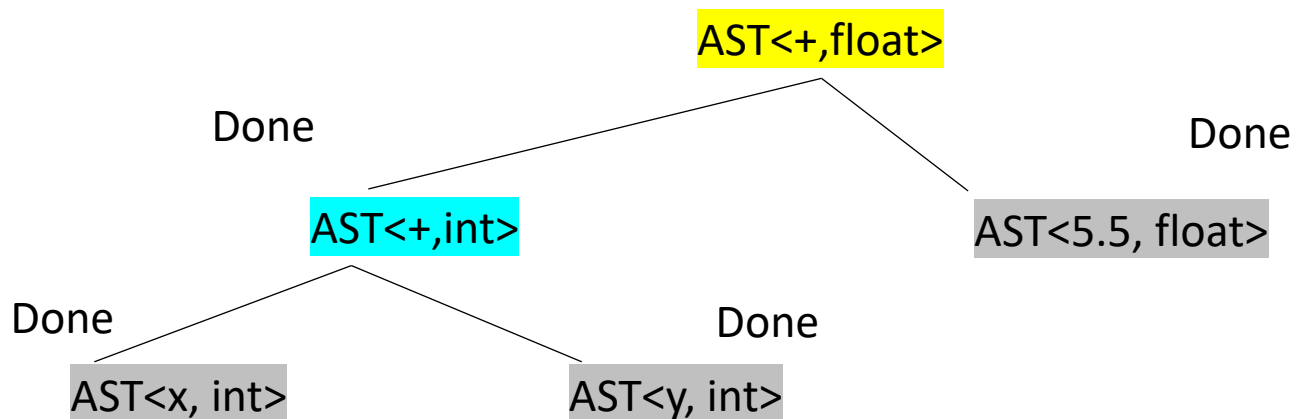
```
    if n.left child type is NOT the same as n type:
```

```
        conv = get conversion AST node
```

```
        conv.child = left child
```

```
        set n.left_child to = conv
```

this will need to be done for both children



New type of AST nodes: unary operators

```
class ASTUnOpNode(ASTNode):  
    def __init__(self, child):  
        self.child = child  
  
class ASTIntToFloatNode(ASTUnOpNode):  
    def __init__(self, child):  
        super().__init__(child)  
  
class ASTFloatToIntNode(ASTUnOpNode):  
    def __init__(self, child):  
        super().__init__(child)
```

```
from enum import Enum
```

```
class Types(Enum):
```

```
    INT = 1
```

```
    FLOAT = 2
```

what types are these nodes?

New type of AST nodes: unary operators

```
class ASTUnOpNode(ASTNode):
```

```
    def __init__(self, child):
```

```
        self.child = child
```

```
class ASTIntToFloatNode(ASTUnOpNode):
```

```
    def __init__(self, child):
```

```
        super().__init__(child)
```

```
class ASTFloatToIntNode(ASTUnOpNode):
```

```
    def __init__(self, child):
```

```
        super().__init__(child)
```

```
from enum import Enum
```

```
class Types(Enum):
```

```
    INT = 1
```

```
    FLOAT = 2
```

what types are these nodes?

New type of AST nodes: unary operators

```
class ASTUnOpNode(ASTNode):
```

```
    def __init__(self, child):
```

```
        self.child = child
```

```
class ASTIntToFloatNode(ASTBinUnNode):
```

```
    def __init__(self, child):
```

```
        self.set_type(Types.FLOAT)
```

```
        super().__init__(child)
```

```
class ASTFloatToIntNode(ASTBinUnNode):
```

```
    def __init__(self, child):
```

```
        self.set_type(Types.INT)
```

```
        super().__init__(child)
```

```
from enum import Enum
```

```
class Types(Enum):
```

```
    INT = 1
```

```
    FLOAT = 2
```

what types are these nodes?

We can go further than just checking to ensure our children are the right type

New type of AST nodes: unary operators

```
class ASTUnOpNode(ASTNode):
```

```
    def __init__(self, child):
```

```
        self.child = child
```

```
class ASTIntToFloatNode(ASTBinUnNode):
```

```
    def __init__(self, child):
```

```
        self.set_type(Types.FLOAT)
```

```
        assert(child.get_type() == Types.INT)
```

```
        super().__init__(child)
```

```
class ASTFloatToIntNode(ASTBinUnNode):
```

```
    def __init__(self, child):
```

```
        self.set_type(Types.INT)
```

```
        assert(child.get_type() == Types.FLOAT)
```

```
        super().__init__(child)
```

```
def type_conversion(n):
```

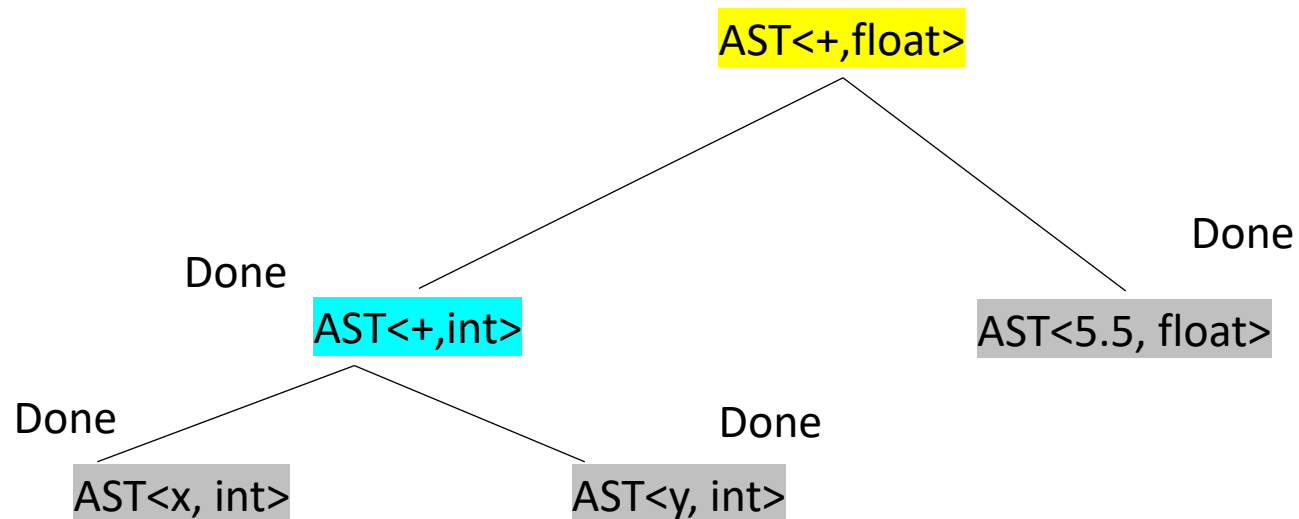
```
    if n.left child type is NOT the same as n type:
```

```
        conv = get conversion AST node
```

```
        conv.child = left child
```

```
        set n.left_child to = conv
```

AST<int2float, float>



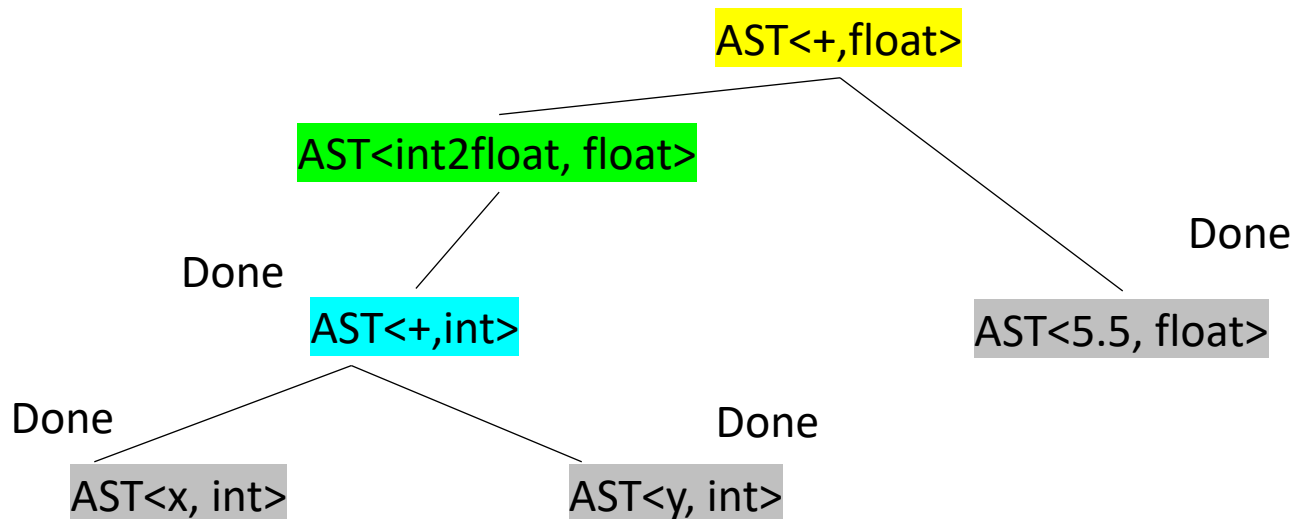
```
def type_conversion(n):
```

```
    if n.left child type is NOT the same as n type:
```

```
        conv = get conversion AST node
```

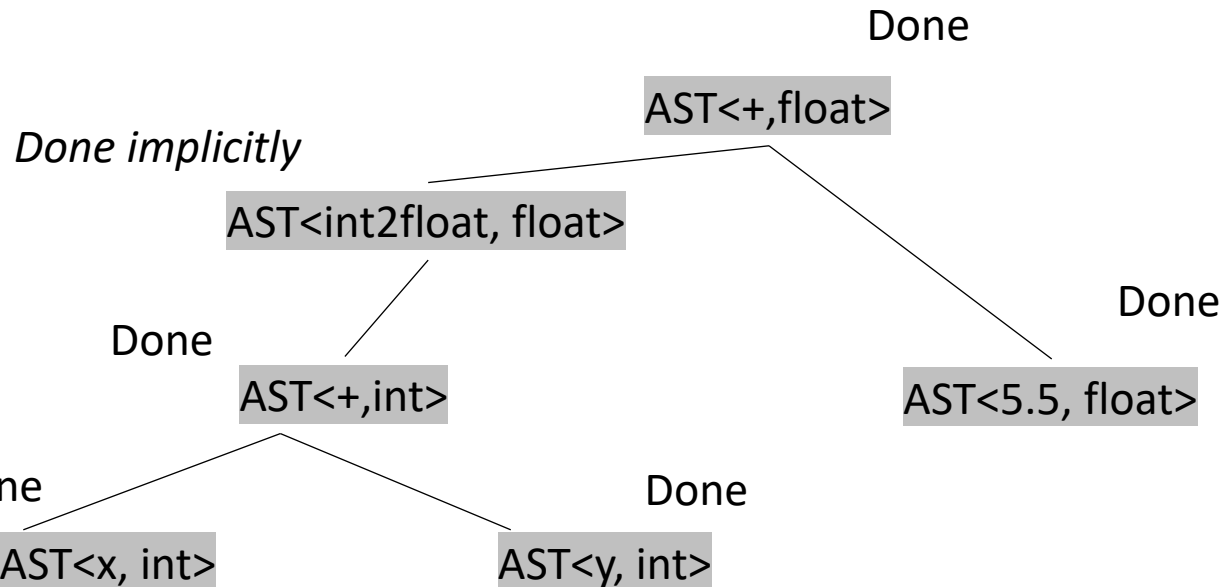
```
        conv.child = left child
```

```
        set n.left_child to = conv
```



Type inference

```
int x;  
int y;  
float w;  
w = x + y + 5.5
```



```
def type_inference(n):
```

```
    case split on type of n:
```

```
        if n is a leaf node:  
            return n.get_type()
```

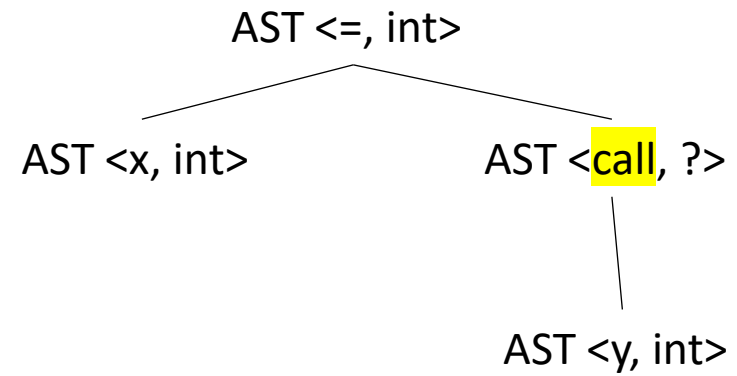
```
        if n is a bin op node:  
            do type inference on children  
            t = lookup type from table  
            set n type to t  
            do any required type conversions  
            return t
```

Done

Topic: Functions

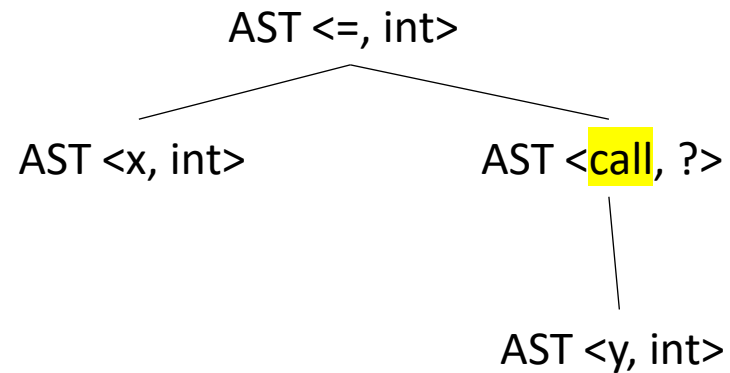
How are functions handled?

```
int x;  
int y;  
x = sqrt(y)
```



How are functions handled?

```
int x;  
int y;  
x = sqrt(y)
```



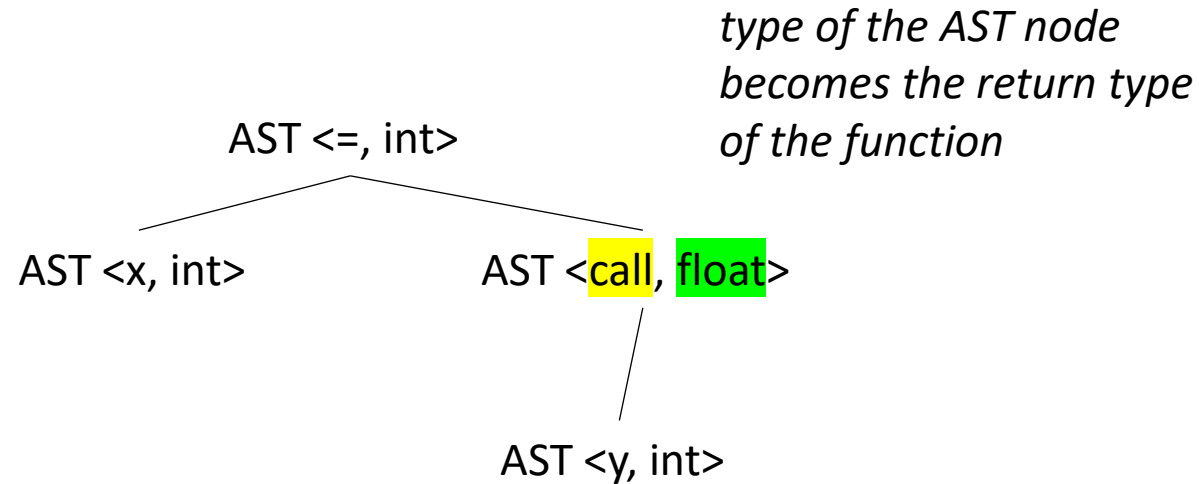
requires a function specification,
using in the .h file:

```
float sqrt(float x) ;
```

stored in the symbol table before type checking - think about C. you have to declare a function before you use it

How are functions handled?

```
int x;  
int y;  
x = sqrt(y)
```



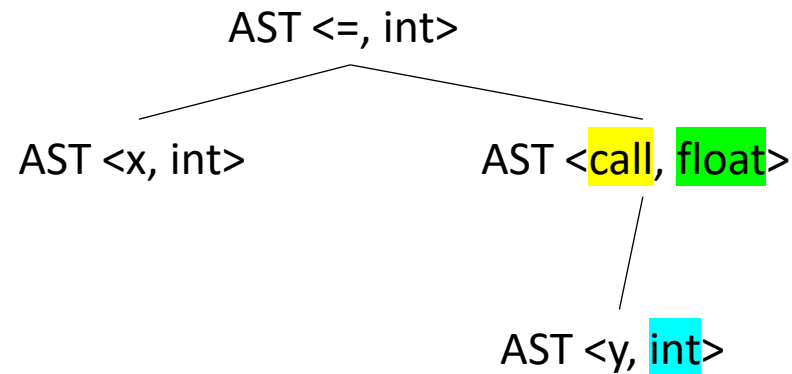
requires a function specification,
using in the .h file:

```
float sqrt(float x);
```

stored in the symbol table before type checking - think about C. you have to declare a function before you use it

How are functions handled?

```
int x;  
int y;  
x = sqrt(y)
```



type inference must make sure arguments match types

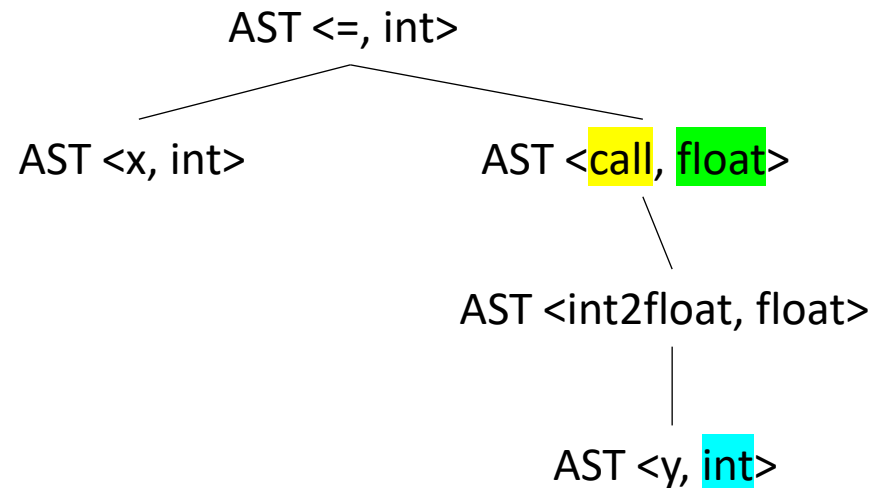
requires a function specification,
using in the .h file:

```
float sqrt(float x) ;
```

stored in the symbol table before type checking - think about C. you have to declare a function before you use it

How are functions handled?

```
int x;  
int y;  
x = sqrt(y)
```



requires a function specification,
using in the .h file:

```
float sqrt(float x) ;
```

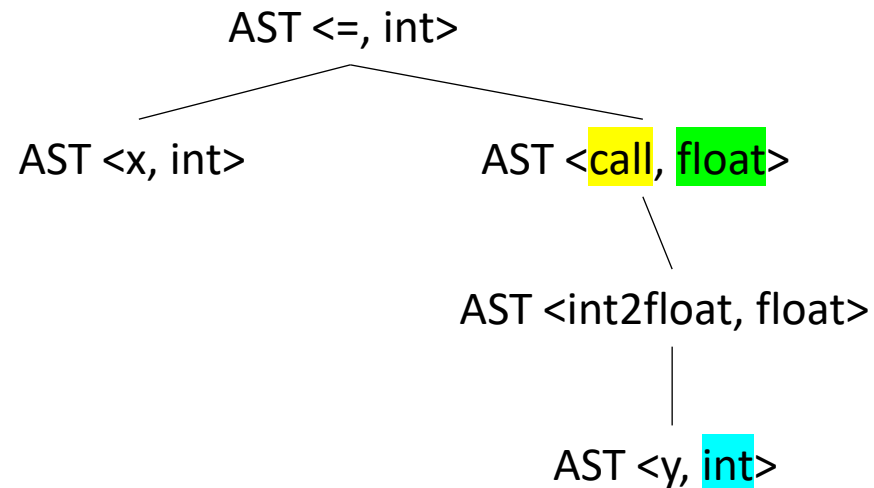
*type inference must make sure
arguments match types*

stored in the symbol table before type checking - think about C. you have to declare a function before you use it

How are functions handled?

```
int x;  
int y;  
x = sqrt(y)
```

How would type inference finish this?



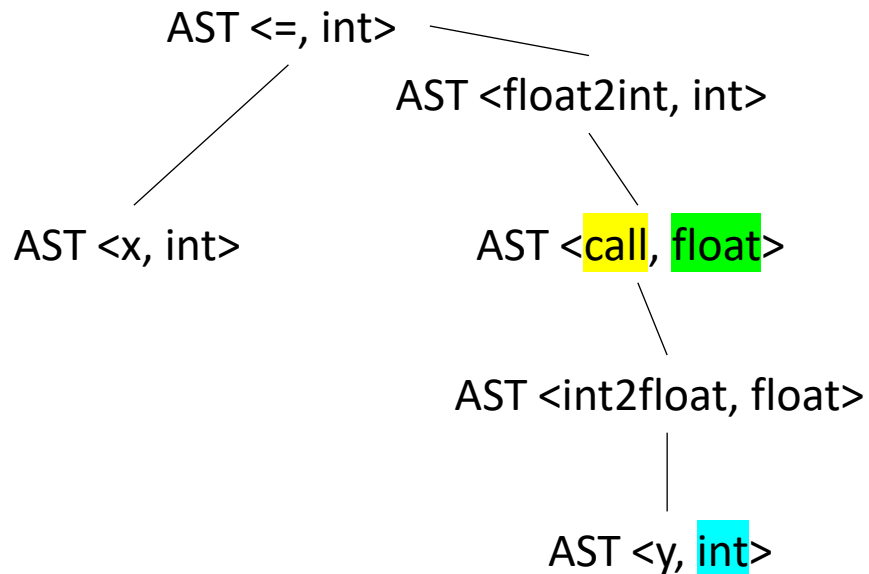
requires a function specification,
using in the .h file:

```
float sqrt(float x) ;
```

stored in the symbol table before type checking - think about C. you have to declare a function before you use it

How are functions handled?

```
int x;  
int y;  
x = sqrt(y)
```



requires a function specification,
using in the .h file:

```
float sqrt(float x);
```

stored in the symbol table before type checking - think about C. you have to declare a function before you use it

*How would type inference finish this?
remember that assignment converts to
the lhs type*

*Binding to the dummy parameter is a
type of assignment.*

Topic: More on Types

What about floats to ints?

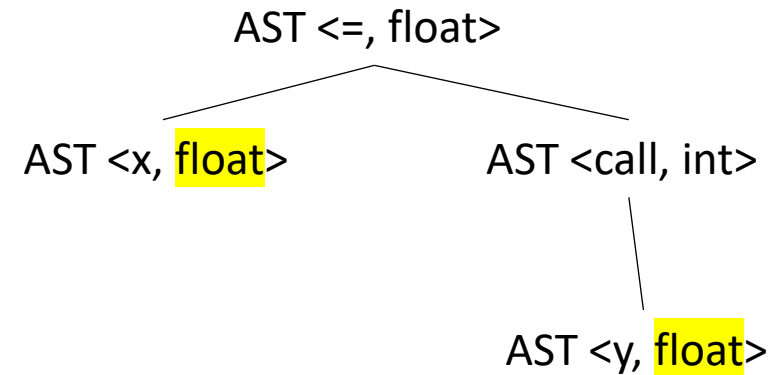
```
int int_sqrt(int input);
```

```
float x;
```

```
float y;
```

```
x = int_sqrt(y)
```

Does this compile?



What about floats to ints?

```
int int_sqrt(int input);
```

```
float x;
```

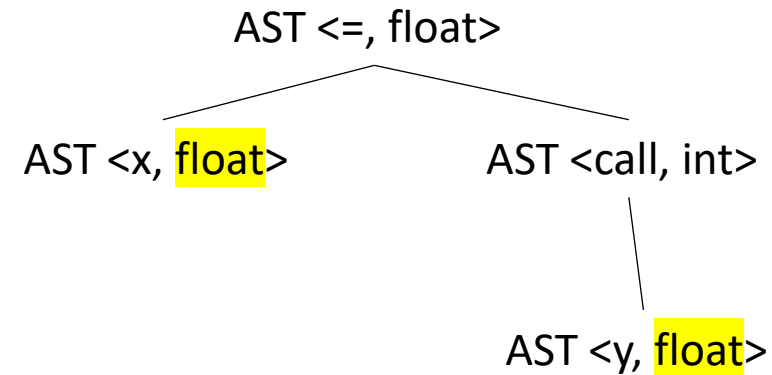
```
float y;
```

```
x = int_sqrt(y)
```

Does this compile? Yes!

In this case the compiler will convert floats to an int.

Is that the right choice? ...



What about floats to ints?

```
int int_sqrt(int input);
```

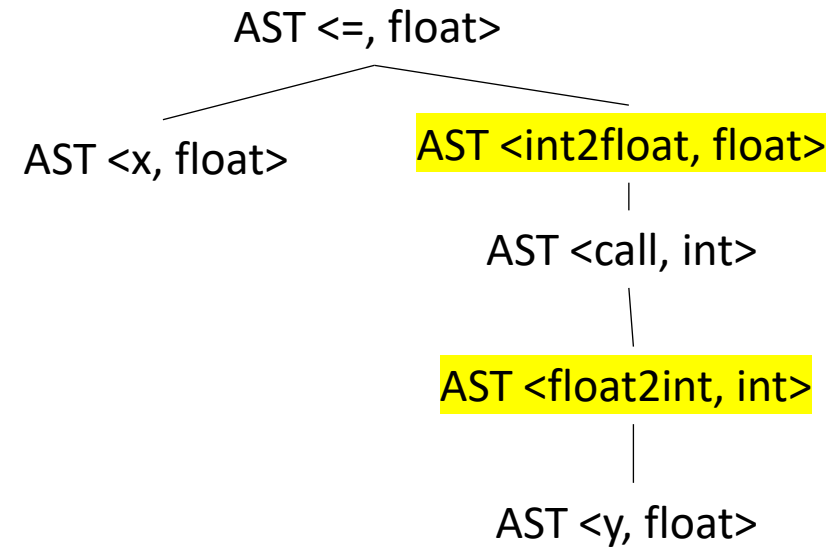
```
float x;
```

```
float y;
```

```
x = int_sqrt(y)
```

Does this compile? Yes!

*In this case the compiler will convert floats to an int.
Is that the right choice? ...*



Discussion

- Many languages (and styles) state that the programmer extends the type system through functions
- Other languages allow operator overloading
 - Controversial design pattern
 - But it can be really nice (e.g. it is used extensively in LLVM internals)

```

class Complex {
private:
    float real;
    float imag;
public:
    // Constructor to initialize real and imag to 0
    Complex() : real(0), imag(0) {}

    // Overload the + operator
    Complex operator + (const Complex& obj) {
        Complex temp;
        temp.real = real + obj.real;
        temp.imag = imag + obj.imag;
        return temp;
    }
}

```

Table for *plus* binary ops

left child	right child	result
int	int	int
int	float	float
float	int	float
float	float	float
Complex	Complex	Complex

```

class Complex {
private:
    float real;
    float imag;
public:
    // Constructor to initialize real and imag to 0
    Complex() : real(0), imag(0) {}

    // Overload the + operator
    Complex operator + (const Complex& obj) {
        Complex temp;
        temp.real = real + obj.real;
        temp.imag = imag + obj.imag;
        return temp;
    }

    Complex operator + (const float& i) {
        Complex temp;
        temp.real = real + i;
        temp.imag = imag;
        return temp;
    }
}

```

Table for *plus* binary ops

left child	right child	result
int	int	int
int	float	float
float	int	float
float	float	float
Complex	Complex	Complex

```

class Complex {
private:
    float real;
    float imag;
public:
    // Constructor to initialize real and imag to 0
    Complex() : real(0), imag(0) {}

    // Overload the + operator
    Complex operator + (const Complex& obj) {
        Complex temp;
        temp.real = real + obj.real;
        temp.imag = imag + obj.imag;
        return temp;
    }

    Complex operator + (const float& i) {
        Complex temp;
        temp.real = real + i;
        temp.imag = imag;
        return temp;
    }
}

```

Table for *plus* binary ops

left child	right child	result
int	int	int
int	float	float
float	int	float
float	float	float
Complex	Complex	Complex
Complex	float	Complex

We can add extra rows

Type systems finished

- Defined what a type system is and discussed various different design decisions
 - static vs. dynamic, choice of primitive types, size of primitive types
- Implemented type inference parameterized by type conversion tables on an AST.
 - identified common conversions (int to float) and when the opposite can happen
- Discussed how programmers can extend the type system
 - function calls
 - operator overloading